

Implementation and Analysis of the Diffie–Hellman Key Exchange

Nanyang Technological University, Singapore

Abstract. This report presents the implementation and analysis of the classical Diffie–Hellman key exchange protocol as part of the SE6003 Cryptography course project. The objective is to demonstrate secure key establishment over an insecure channel, evaluate correctness of the implementation, and discuss critical security considerations.

Keywords: Diffie–Hellman · Key Exchange · Cryptography · Discrete Logarithm Problem

1 Introduction

In modern communication systems, the volume and throughput of transmitted data have increased drastically compared to the early days of the Internet. As a consequence, the demand for efficient and secure encryption mechanisms is greater than ever. While symmetric-key cryptography provides excellent performance for encrypting large amounts of data, it cannot securely establish keys over an untrusted channel. In contrast, asymmetric-key cryptography can securely encrypt data and establish keys over an insecure network, but it is computationally inefficient for encrypting large amounts of data directly [8,9].

Modern secure communication protocols solve this challenge by combining both approaches in a hybrid manner. The Diffie–Hellman key exchange protocol is a classical and foundational method for establishing a shared symmetric key between two parties over an insecure network, without transmitting the key itself [5]. This shared key can then be used to perform efficient symmetric encryption, providing both confidentiality and performance.

This report provides:

- a conceptual background of the Diffie–Hellman protocol
- the mathematical foundations underlying its security
- implementation details of the protocol
- a security analysis of the approach
- test results and conclusions

2 Background

2.1 The Diffie–Hellman Problem

The security of the Diffie–Hellman key exchange relies on the computational difficulty of the **Discrete Logarithm Problem (DLP)** [5,7]. Given integers g , h , and a large prime modulus p , the DLP asks for an integer x such that

$$g^x \equiv h \pmod{p}. \quad (1)$$

For appropriately chosen parameters, computing such an x is believed to be computationally infeasible[5,7].

Diffie–Hellman makes use of this assumption by selecting a large prime p and a generator g of a cyclic multiplicative group modulo p . If g generates a group of sufficiently large order, it becomes computationally infeasible for an adversary to brute-force the exponent x given only g and $g^x \bmod p$. This property forms the foundation of the protocol’s security.

2.2 Protocol Description

Alice and Bob perform the Diffie–Hellman key exchange as follows. They first agree on a large prime p and a generator g of a cyclic group modulo p .

1. Alice chooses a private key a and computes her public value

$$A = g^a \bmod p,$$

which she sends to Bob.

2. Bob chooses a private key b and computes his public value

$$B = g^b \bmod p,$$

which he sends to Alice.

3. Upon receiving each other’s public values, both compute the shared secret

$$S = g^{ab} \bmod p = B^a \bmod p = A^b \bmod p.$$

An eavesdropper, Charlie, can observe the public values A and B , but without knowledge of the private keys a or b , he cannot efficiently compute the shared secret S , due to the hardness of the DLP.

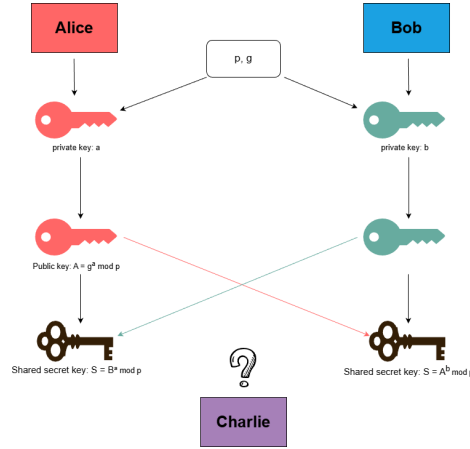


Fig. 1. Illustration of the Diffie–Hellman key exchange protocol.

3 Methodology and Implementation

3.1 Tools and Programming Environment

We implemented a primitive version of the Diffie–Hellman key exchange in C++. The implementation includes prime generation of variable size, selection of a generator, key exchange, and basic communication between two parties over a simulated insecure network. We used the `OpenSSL`[2] library for cryptographically secure random number generation[3], hashing, and AES encryption. The GNU Multiple Precision Arithmetic Library (`GMP`)[1] is used for arbitrary-precision arithmetic, allowing us to handle large integers such as the prime modulus p that cannot be contained by standard C++ data types.

3.2 Prime Generation

The ideal prime for Diffie–Hellman is a *safe prime* $p = 2q + 1$, where q is also prime. This ensures that the multiplicative group $\mathbb{Z}_p^*$ contains a large prime-order subgroup (see Section 3.3), making the discrete logarithm problem hard. In our implementation, we generate a single safe prime using `OpenSSL`'s `RAND_bytes` and `GMP` for arbitrary-precision arithmetic.

For efficiency, this prime is reused for all communications in the simulation. Generating a new 2048-bit safe prime for every key exchange is computationally expensive, and reusing a single safe prime does not compromise security in this context, as the security of Diffie–Hellman relies on the secrecy and randomness of the private exponents a and b , rather than the uniqueness of the prime.

3.3 Generator Selection

Once a safe prime p has been chosen, the next step is to select a generator g of a suitable subgroup of $\mathbb{Z}_p^*$. For a safe prime of the form $p = 2q + 1$, the multiplicative group $\mathbb{Z}_p^*$ has order

$$|\mathbb{Z}_p^*| = p - 1 = 2q.$$

By Lagrange's Theorem, any subgroup $H \subseteq \mathbb{Z}_p^*$ must have an order dividing $|\mathbb{Z}_p^*|$. Since $2q$ has only the factors $\{1, 2, q, 2q\}$, these are the only possible subgroup orders. The subgroups of order 1 and 2 are trivial and provide no security, leaving only the prime-order subgroup of size q and the full group of size $2q$.

In our implementation, we choose g to be a generator of the prime-order subgroup of size q . We avoid using the full group of size $2q$ because it contains an element of order 2, which deterministically leaks one bit of information from the shared secret — a property well-known in the cryptography community (see Section 4.5).

A value g belongs to the prime-order subgroup if and only if

$$g^q \equiv 1 \pmod{p}.$$

Since q is prime and the subgroup structure of $\mathbb{Z}_p^*$ is fully determined by the factors of $2q$, this condition is sufficient to ensure that g has order exactly q .

In our implementation, candidate values for g are examined sequentially over the interval $[2, p - 2]$. For each candidate, we evaluate the above criterion. Any value satisfying it is accepted as a generator of the subgroup of order q . Once such a generator is found, the search terminates and the selected g is used for all subsequent protocol operations.

3.4 Key Derivation and Encryption

After computing the shared secret

$$S = g^{ab} \bmod p,$$

both Alice and Bob derive a symmetric key for encryption. To convert the large integer S into a fixed-size key suitable for **AES**, S is hashed using **SHA-256**, a cryptographically secure hash, via the **OpenSSL** library. The resulting 256-bit value is used as the AES key for encrypting and decrypting subsequent messages.

All messages are encrypted using AES in CBC (Cipher Block Chaining) mode. For each message, a fresh secure random **initialisation vector (IV)** is generated using **OpenSSL's RAND_bytes**. This ensures that identical plaintext messages produce different ciphertexts, preventing patterns from being observable to an adversary. The IV is transmitted along with the ciphertext so that the recipient can decrypt the message correctly.

This approach allows Alice and Bob to securely exchange encrypted messages over the insecure network while using a fixed-length AES key derived from the shared Diffie–Hellman secret.

3.5 Key Implementation Snippets

We highlight a few critical implementation functions:

Listing 1.1. Modular exponentiation using square-and-multiply

```
1 inline mpz_class ModExpo(const mpz_class& base, const
    mpz_class& exponent, const mpz_class& mod) { ... }
```

Listing 1.2. Generate a cryptographically secure random number of given byte length

```
1 inline mpz_class GenerateRandomNumber(const int& numBytes) {
    ... }
```

Listing 1.3. Find a generator of prime-order subgroup

```
1 inline mpz_class FindGenerator(const mpz_class& p, const
    mpz_class& q) { ... }
```

Listing 1.4. Generate a safe-prime (default at 2048-bit prime)

```
1 inline mpz_class GenSafePrime(const int numBit = 2048) { ...
    }
```

Listing 1.5. Generate a cryptographically secure private key

```
1 inline mpz_class GeneratePrivateKey(const mpz_class& q) { ...
    }
```

3.6 Program Execution and Simulation

The main program simulates two users, Alice and Bob, communicating over a simulated insecure network. Each user is represented by a `User` object connected to a shared `Network` object, allowing messages to be sent in both unencrypted and encrypted form.

Initially, unencrypted messages are exchanged to demonstrate basic communication. All network messages are logged in raw form, simulating an environment where an eavesdropper could observe them. Each user also maintains a local log of messages they send and receive, including unencrypted and decrypted messages. For simplicity, key exchange messages are not logged by users.

Next, Alice and Bob perform a Diffie–Hellman key exchange to derive a shared symmetric key. Subsequent messages are encrypted with AES using this key.

This simulation demonstrates the full lifecycle of the protocol: unencrypted messaging, secure key exchange, encrypted communication, and message logging for analysis.

4 Results and Evaluation

In this section, we present the results of our Diffie–Hellman key exchange simulation, highlighting the communication flow between Alice and Bob over the network. We evaluate both unencrypted and encrypted messages from the perspective of the network, as well as the local logs of Alice and Bob.

4.1 Network Perspective

The network logs all messages transmitted between users. Figure 2 shows the message flow:

```

1 ===== NETWORK =====
2 [Alice]: hello bob from alice
3 [Bob]: hello alice from bob
4
5 [Bob]: 16c93ae85e8b18afc27409... (truncated)
6 [Alice]: 6b70c41dab21332b4cf3d... (truncated)
7
8 [Alice]: [ENCRYPTED (base64)]
9 Rr5MlmFbIv+gBH3dyFjD... (truncated)
10 IV: c54f9cd41cc244f19e5a... (truncated)
11 [Bob]: [ENCRYPTED (base64)]
12 Uv9cjLfNwQuMft8+mTNE... (truncated)
13 IV: 9b63c7becf3bd13f967c... (truncated)

```

Fig. 2. Network log showing unencrypted messages (line 2-3), key exchange (line 5-6), and encrypted communication (line 8-13) (ciphertext encoded to base64, public key hex and ciphertext truncated for readability).

4.2 Alice's and Bob's Perspective

Alice and Bob maintain a local log of all messages received. Key exchange messages are not logged locally. Figure 3 shows Alice's view, and Figure 4 shows Bob's view:

```

1 ===== Alice =====
2 hello alice from bob
3 hello alice this is encrypted from bob

```

Fig. 3. Alice's message log showing her received messages. Key exchange messages are not logged.

```

1 ===== Bob =====
2 hello bob from alice
3 hello bob this is encrypted from alice

```

Fig. 4. Bob’s message log showing his received messages. Key exchange messages are not logged.

4.3 Evaluation

The logs illustrate the security and functionality of our implementation:

- **Before the key exchange:** All observers can see the raw plaintext messages exchanged between Alice and Bob.
- **During the key exchange:** Observers can only see the public components, Alice’s $A = g^a \bmod p$ and Bob’s $B = g^b \bmod p$, as shown in Figure 2. Without knowledge of the private keys a or b , there is no efficient way to compute the shared secret S [5,7].
- **After the key exchange:** Alice and Bob can securely communicate. All subsequent messages are encrypted using AES in CBC mode, with a fresh IV for each message. Observers can only see the ciphertexts and the associated IVs, shown in Figure 2.

This demonstrates that the Diffie–Hellman key exchange successfully establishes a shared secret that enables secure communication over an insecure network.

4.4 Possible Attacks & Vulnerabilities

Man-in-the-Middle (MITM) Attack The basic Diffie–Hellman protocol, as implemented in this project, provides *no authentication*. This means it cannot defend against an active adversary who can intercept and modify messages [4,6].

Suppose an adversary, Mallory, is able to intercept the communication channel. When Alice sends her public value $A = g^a \bmod p$ to Bob, Mallory blocks it and replaces it with his own value $Z = g^z \bmod p$. Mallory performs the same substitution when Bob sends his public value $B = g^b \bmod p$.

As a result:

- Alice believes she is receiving Bob’s public value, but she actually receives Mallory’s Z .
- Bob believes he is receiving Alice’s public value, but again receives Z .

Alice and Mallory then compute a shared secret $S_{AM} = g^{az} \bmod p$, while Bob and Mallory compute a different shared secret $S_{BM} = g^{bz} \bmod p$. Alice and Bob both think they share a secret with each other, but in reality, each shares a secret only with Mallory.

This enables Mallory to fully compromise the communication:

- When Alice sends an encrypted message intended for Bob, Mallory can decrypt it using S_{AM} .
- Mallory then re-encrypts the message using S_{BM} and forwards it to Bob.

From Alice and Bob’s perspective, everything appears normal—they can send and receive correctly encrypted messages. However, Mallory is able to read and modify all communication without detection.

This attack highlights why Diffie–Hellman must be paired with authentication mechanisms (e.g., digital signatures or certificates) in real-world secure protocols such as TLS [4,6].

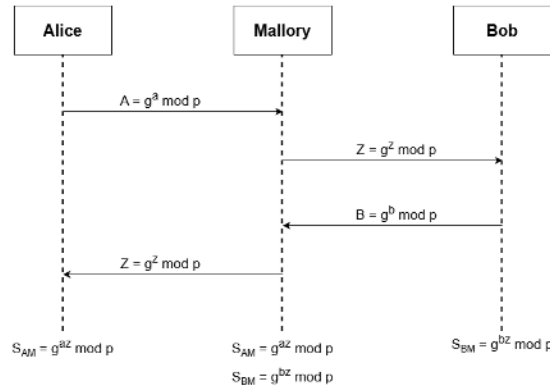


Fig. 5. Man-in-the-Middle attack on the unauthenticated Diffie–Hellman key exchange. Mallory transparently establishes two shared secrets, allowing interception, decryption, and re-encryption of all messages.

4.5 One-Bit Leakage from Full Group Selection

If the full multiplicative group $\mathbb{Z}_p^*$ of order $2q$ (with q prime) is used in Diffie–Hellman, the structure of the group enables an adversary to recover the least significant bit (LSB) of any secret exponent. Given a public key $A = g^a \bmod p$, the adversary can compute

$$A^q \bmod p.$$

Since g is chosen to generate the full group of order $2q$, it satisfies

$$g^{2q} \equiv 1 \pmod{p}.$$

This implies that

$$g^q \equiv \pm 1 \pmod{p},$$

because any element whose square is 1 must be either 1 or -1 . However, g^q cannot be 1, as that would imply that the order of g divides q , contradicting the fact that $\text{ord}(g) = 2q$. Therefore,

$$g^q \equiv -1 \pmod{p}.$$

Now consider the parity of the secret exponent a :

- If a is even, write $a = 2k$. Then

$$A^q = g^{aq} = g^{2kq} = (g^{2q})^k \equiv 1^k \equiv 1 \pmod{p}.$$

- If a is odd, write $a = 2k + 1$. Then

$$A^q = g^{(2k+1)q} = g^{2kq} \cdot g^q \equiv 1 \cdot (-1) \equiv -1 \pmod{p}.$$

Thus, an adversary can immediately determine the least significant bit of a :

$$A^q \equiv \begin{cases} 1 & \text{if } a \text{ is even (LSB} = 0\text{)}, \\ -1 & \text{if } a \text{ is odd (LSB} = 1\text{)}. \end{cases}$$

It is worth noting that, although the use of the full group leaks one bit of information about the secret exponent, in practice this leakage is not a serious concern when sufficiently large key sizes are used. For typical Diffie–Hellman parameters (e.g., 2048-bit primes), knowing a single bit of the exponent does not materially reduce the effort required for an adversary to compute the full secret. Nevertheless, for formal security proofs and to avoid even minimal deterministic leakage, restricting to the prime-order subgroup of size q is considered best practice.

5 Conclusion

In this report, we presented a complete implementation of the classical Diffie–Hellman key exchange, detailing the methodology for selecting the prime and generator and the security considerations behind these choices.

We also examined potential attacks, such as the man-in-the-middle attack, highlighting that the unauthenticated protocol is vulnerable to active adversaries. This demonstrates that while Diffie–Hellman is highly effective for establishing shared secrets over insecure channels, it must be combined with authentication mechanisms to ensure practical security.

Overall, our implementation illustrates that the Diffie–Hellman key exchange remains a foundational tool in modern secure communication, provided that proper parameters are chosen and additional security measures are applied for real-world deployment.

References

1. Gnu multiple precision arithmetic library (gmp). <https://gmplib.org>
2. Openssl cryptography and ssl/tls toolkit. <https://www.openssl.org>
3. Rand.bytes — openssl 3.1 manual. https://docs.openssl.org/3.1/man3/RAND_bytes/#synopsis

4. Blake-Wilson, S., Menezes, A.: Authenticated diffie-hellman key agreement protocols. In: *Advances in Cryptology - SAC '98: Selected Areas in Cryptography*, Lecture Notes in Computer Science, vol. 1556, pp. 339–359. Springer (1998). https://doi.org/10.1007/3-540-48892-8_26
5. Diffie, W., Hellman, M.: New directions in cryptography. *IEEE Transactions on Information Theory* **22**(6), 644–654 (1976). <https://doi.org/10.1109/TIT.1976.1055638>
6. Khader, A.S., Lai, D.: Preventing man-in-the-middle attack in diffie-hellman key exchange protocol. In: *2015 22nd International Conference on Telecommunications (ICT)*. pp. 204–208 (2015). <https://doi.org/10.1109/ICT.2015.7124683>
7. McCurley, K.S.: The discrete logarithm problem. In: Pomerance, C. (ed.) *Cryptology and Computational Number Theory, Proceedings of Symposia in Applied Mathematics*, vol. 42, pp. 49–74. American Mathematical Society (1990), <http://www.mccurley.org/papers/dlog.pdf>
8. Santoso, P.P., Rilvani, E., Trisnawan, A.B., Adiyarta, K., Napitupulu, D., Sutabri, T., Rahim, R.: Systematic literature review: comparison study of symmetric key and asymmetric key algorithm. *IOP Conference Series: Materials Science and Engineering* **420**(1), 012111 (sep 2018). <https://doi.org/10.1088/1757-899X/420/1/012111>, <https://doi.org/10.1088/1757-899X/420/1/012111>
9. Shah, A.M., Gor, A.: Comparative analysis of security algorithms. *International Journal of Informative & Futuristic Research (IJIFR)* **12**(10), 800–812 (2025), <https://www.ijifr.org/pdfs/18-06-2025800IJIFR-V12-E10-005.pdf>